Criterion A: Problem Specification

Part 1: Problem Scenario

The intended user of this solution is a **school librarian responsible for managing a physical book collection of increasing size**. In the current situation, book records are often stored using simple linear structures such as spreadsheets or unsorted lists. As the number of books grows, routine operations such as searching for a book by ISBN, inserting new records, or removing outdated entries become increasingly inefficient. These systems also lack guaranteed performance, meaning that search time may degrade significantly as more data is added.

The core cause of the problem is the reliance on **inefficient data storage methods that do not scale well** and do not maintain book records in a sorted order. While basic data structures can store information, they do not provide consistent performance for frequent insert, delete, and search operations. This directly affects the librarian, particularly during busy periods, as delays in locating or updating records reduce overall efficiency and usability.

To address this problem, a computational solution is required that can **store book records in a structured, ordered, and scalable manner**, while remaining easy to use for a non-technical user. The system must allow the librarian to manage book information, track availability status, and quickly locate specific records through an intuitive interface. These requirements form the basis for the success criteria used to evaluate the final product.

Part 2: Computational Context

The solution is implemented as a **stand-alone C++ application with both a command-line interface and a graphical user interface (GUI)**. A stand-alone application is appropriate because the system is designed for **single-user local operation** and does not require internet connectivity. C++ was chosen due to its performance efficiency and its suitability for implementing pointer-based data structures. A **red-black tree** is used as the core data structure to ensure that all book records remain balanced and sorted, guaranteeing efficient O(log n) insert, delete, and search operations even for large datasets. SFML is used to provide a responsive and user-friendly graphical interface.

Part 3: Success Criteria

The solution will be considered successful if it meets the following criteria:

1. The system allows the user to add book records containing ISBN, title, author, year, and availability status.
2. All book records are stored in sorted order based on ISBN.
3. The user can search for a book by ISBN and receive the correct record within one second.
4. The system allows existing book records to be removed using their ISBN.
5. Duplicate ISBN entries are rejected with an appropriate error message.
6. The user can check out and return books, with availability status updated correctly.
7. The system can display all books currently stored in the library in sorted order.
8. The system remains stable and responsive after at least 1,000 book records are inserted.
9. Invalid user inputs do not cause the program to crash and are handled with warning messages.